AG TECHNOLOGIES

Architecture de Services Partagés

Cahier des Charges Technique

Service de Géolocalisation Centralisé

Geolocation Service — AGT-Geo

Version	1.2
Date	Avril 2026
Statut	Implementation-ready
Classification	Confidentiel
Auteur	Équipe Architecture
Dépendances	Auth v2.1, Users v1.0, Notification v1.2

Document confidentiel — Réservé à l'usage interne d'AG Technologies

Historique des Révisions

Ver.	Date	Auteur	Description
1.0	2026-04-05	Équipe Archi.	Création initiale. Position temps réel (WebSocket + REST). Redis live + PostGIS historique. Proximité. Géofencing event-driven scalable. Historique trajets. Multi-provider ETA/géocodage (adapter pattern). Fréquence configurable par plateforme. Batch writes PostGIS. Diagramme PlantUML en annexe.
1.1	2026-04-05	Équipe Archi.	Post-audit : dual Redis formalisé (index GEO + payload entité). Autorisation WebSocket verrouillée. RGPD compatible schéma (anonymisation irréversible). Machine à états trips. NF-15 corrigé. Rôle Users clarifié. Invalidation cache geofence sur soft delete. Modèle de présence/offline.
1.2	2026-04-05	Équipe Archi.	Version finale : statut *Implementation-ready*.

Table des matières

1 Introduction **4**
1.1 Périmètre . . . 4
1.2 Hors périmètre . . . 4

2 Contexte & Problématique **4**
2.1 Contexte . . . 5
2.2 Problématique . . . 5

3 Vision & Objectifs **5**
3.1 Vision . . . 5
3.2 Objectifs . . . 5

4 Parties Prenantes & Utilisateurs **5**

5 Besoins Fonctionnels **5**
5.1 Position Temps Réel . . . 6
5.2 Recherche de Proximité . . . 7
5.3 Géofencing . . . 7
5.4 Trajets . . . 8
5.5 Distance & ETA . . . 8
5.6 Géocodage . . . 9
5.7 Administration . . . 9

6 Besoins Non-Fonctionnels **10**

7 Architecture Technique **10**
7.1 Vue d'ensemble . . . 10
7.2 Stack Technique . . . 11
7.3 Dépendances & Règles d'Architecture . . . 11
7.4 Stratégie de scalabilité . . . 12
7.5 Modèle de présence / offline (v1.1) . . . 12
7.6 Diagramme d'architecture . . . 12

8 Modélisation **13**
8.1 Diagramme de Classes . . . 13
8.2 MLD — Modèle Logique de Données . . . 13
8.2.1 Table `tracked_entities` . . . 13
8.2.2 Table `position_history` . . . 13
8.2.3 Table `trips` . . . 14
8.2.4 Table `geofences` . . . 14
8.2.5 Table `geofence_events` . . . 15
8.2.6 Table `platform_geo_configs` . . . 15

9 Contrats d'API **15**
9.1 Health Check . . . 15
9.1.1 GET /geo/health . . . 15
9.2 Position (REST) . . . 16
9.2.1 POST /geo/positions . . . 16
9.2.2 GET /geo/positions/{entityId} . . . 16
9.3 Position (WebSocket) . . . 17
9.3.1 Connexion WebSocket . . . 17

9.4 Proximité . . . 17
9.4.1 GET /geo/proximity . . . 17
9.5 Trajets . . . 18
9.5.1 POST /geo/trips . . . 18
9.5.2 POST /geo/trips/{tripId}/end . . . 18
9.5.3 GET /geo/trips/{tripId} . . . 19
9.5.4 GET /geo/trips . . . 19
9.6 Géofences . . . 19
9.6.1 POST /geo/geofences . . . 19
9.6.2 GET /geo/geofences . . . 19
9.6.3 PUT /geo/geofences/{geofenceId} . . . 20
9.6.4 DELETE /geo/geofences/{geofenceId} . . . 20
9.7 Distance & ETA . . . 20
9.7.1 GET /geo/distance . . . 20
9.7.2 GET /geo/eta . . . 20
9.8 Géocodage . . . 20
9.8.1 GET /geo/geocode . . . 20
9.8.2 GET /geo/reverse-geocode . . . 21
9.9 Configuration Plateforme . . . 21
9.9.1 PUT /geo/config/{platformId} . . . 21
9.9.2 GET /geo/config/{platformId} . . . 21
9.10 Administration . . . 21
9.10.1 GET /geo/admin/stats . . . 21
9.10.2 DELETE /geo/by-user/{userId} . . . 22

10 Contrats Inter-Services **22**
10.1 Géolocalisation → Auth (JWT / S2S) . . . 22
10.2 Géolocalisation → Notification (alertes) . . . 22
10.3 Géolocalisation — Événements RabbitMQ sortants . . . 22
10.4 Users → Géolocalisation (Purge RGPD) . . . 23

11 Matrice d'Autorisation **23**

12 Recommandations & Évolutions **23**

1 Introduction

Ce document constitue le cahier des charges du **Service de Géolocalisation Centralisé** (ci-après « Geolocation Service ») destiné à l'écosystème AGT. Il sert de référence unique pour la conception, le développement et les décisions techniques.

Principes fondamentaux — À lire en premier

1. **Dual storage** : Redis pour les positions live (volatile, TTL), PostGIS pour l'historique et les requêtes spatiales (persistant).
2. **Fréquence configurable** : chaque plateforme définit l'intervalle de mise à jour (1s à 60s).
3. **Géofencing event-driven** : vérification à chaque mise à jour de position, pas en polling. Scalable.
4. **Batch writes** : les positions sont batchées avant insertion PostGIS (10–30s) pour absorber la charge.
5. **Multi-provider** : ETA et géocodage abstraits derrière des adapters (Google Maps, OSRM, Nominatim).
6. **Stateless** : chaque instance NestJS est interchangeable. Redis est le point de coordination.

1.1 Périmètre

Le service Géolocalisation couvre : réception et diffusion des positions en temps réel, stockage de l'historique des trajets, recherche de proximité, géofencing (zones avec alertes entrée/sortie), calcul de distance/ETA, et géocodage (adresse ↔ coordonnées).

Principe fondamental

Le service gère les **coordonnées et la logique spatiale**. Il ne connaît aucun concept métier : il ne sait pas ce qu'est un chauffeur, un livreur ou un vendeur. Il gère des `tracked_entities` avec des positions, des trajets et des zones. La plateforme consommatrice donne le sens métier.

1.2 Hors périmètre

- Authentification (déléguée au Service Auth v2.1)
- Profils utilisateur (Service Users v1.0)
- Logique métier (dispatch, affectation de courses, matching vendeur/livreur)
- Affichage cartographique (le frontend utilise Google Maps / Mapbox directement)
- Paiement, abonnement, wallet
- Notification (déléguée au Service Notification v1.2)

Convention d'identité

Dans tout ce document :

- `user_id` désigne `users_auth.id` (= `sub` du JWT)
- `platform_id` désigne un UUID du registre Auth (`platforms.id`)
- `entity_id` désigne l'identifiant de l'entité trackée (= `user_id` dans la plupart des cas)

2 Contexte & Problématique

2.1 Contexte

Plateforme	Statut	Besoin géolocalisation
MboaMove	En développement	Position live chauffeurs, proximité, ETA, suivi de course, historique trajets
AGT-Market	En développement	Position livreurs, suivi livraison, proximité vendeurs
SALMA	Migration prévue	Localisation centres de formation (statique)
Futures	Planifié	Tout besoin de localisation

2.2 Problématique

Sans service centralisé, chaque plateforme implémenterait sa propre gestion GPS, entraînant : duplication des intégrations cartographiques, absence de recherche de proximité performante, pas de géofencing réutilisable, et aucune vision unifiée des données de localisation.

3 Vision & Objectifs

3.1 Vision

Un service **générique et abstrait** qui gère les positions, trajets et zones pour tout l'écosystème. Il ne connaît aucun concept métier : il gère des entités trackées, des coordonnées et des polygones.

3.2 Objectifs

#	Objectif	Indicateur
O1	Position temps réel	Latence $<$ 200ms entre envoi et diffusion
O2	Proximité performante	Trouver les N plus proches en $<$ 100ms
O3	Historique trajets	Stockage et restitution de tout trajet
O4	Géofencing scalable	10 000+ zones actives sans dégradation
O5	Multi-provider	ETA et géocodage abstraits (adapter pattern)
O6	Fréquence configurable	1s à 60s selon la plateforme et le contexte
O7	Scalabilité horizontale	100 000+ entités trackées simultanément

4 Parties Prenantes & Utilisateurs

Rôle	Type	Responsabilité
Product Owner	Interne	Priorisation des fonctionnalités
Équipe Backend	Interne	Développement, intégration providers
Équipe Frontend	Interne	Intégration carte, suivi temps réel
Équipe Mobile	Interne	SDK GPS, envoi positions, battery management
DevOps	Interne	Infrastructure Redis/PostGIS, monitoring
Admin global AGT	Interne	Configuration zones, providers
Admin plateforme	Interne	Géofencing, fréquence, consultation trajets
Utilisateurs finaux	Externe	Partage de position, suivi
Services internes	Interne	Plateformes (dispatch, suivi livraison)

5 Besoins Fonctionnels

5.1 Position Temps Réel

ID	Fonctionnalité	Description	Priorité
F-POS-01	Envoi position (WebSocket)	L'entité envoie sa position via WebSocket. Haute fréquence.	**Urgent**
F-POS-02	Envoi position (REST)	Endpoint REST pour les mises à jour ponctuelles ou batch	**Urgent**
F-POS-03	Stockage live Redis	Dernière position connue stockée dans Redis avec TTL	**Urgent**
F-POS-04	Diffusion live	Les abonnés (plateformes, autres users) reçoivent les mises à jour en temps réel via WebSocket	**Urgent**
F-POS-05	TTL auto-expiration	Si aucune mise à jour pendant X secondes (configurable), l'entité est considérée offline	**Urgent**
F-POS-06	Fréquence configurable	Chaque plateforme définit l'intervalle de mise à jour (défaut : 5s)	**Urgent**
F-POS-07	Batch write PostGIS	Les positions sont accumulées en mémoire/Redis et flushées vers PostGIS en batch (10–30s)	**Urgent**

Architecture dual-storage (v1.1)

Le stockage live Redis repose sur **deux couches complémentaires** :

Couche A — Index GEO par plateforme (proximité) :

- Clé : `geo:index:{platform_id}`
- Contenu : membres = `entity_id`, avec longitude et latitude
- Commandes : `GEOADD`, `GEOSEARCH`, `GEODIST`
- Usage : **exclusivement** pour la recherche de proximité live

Couche B — Payload détaillé par entité :

- Clé : `geo:entity:{platform_id}:{entity_id}`
- Valeur : `{lat, lng, heading, speed, accuracy, tags, entity_type, recorded_at, is_online}`
- TTL : configurable par plateforme (défaut 30s). Expiration = entité offline.
- Usage : lecture de la dernière position détaillée, diffusion WebSocket

Règle : la proximité live n'est **jamais** calculée à partir des clés individuelles d'entité. Elle s'appuie exclusivement sur l'index GEO par plateforme.

Mise à jour atomique : chaque position update met à jour les deux couches dans la même opération (pipeline Redis).

PostGIS (historique) :

- Table `position_history` avec colonne `GEOGRAPHY(Point, 4326)`.
- Index GiST pour les requêtes spatiales.
- Insertion en batch (buffer de 10–30s) pour absorber la charge.
- Partitionnement par mois sur `recorded_at`.

5.2 Recherche de Proximité

ID	Fonctionnalité	Description	Priorité
F-PRX-01	Recherche par rayon	Trouver toutes les entités dans un rayon de N km autour d'un point	**Urgent**
F-PRX-02	N plus proches	Trouver les N entités les plus proches d'un point	**Urgent**
F-PRX-03	Filtre par plateforme	La recherche est scopée par `platform_id`	**Urgent**
F-PRX-04	Filtre par tags	Filtrer par tags métier (ex : `available`, `busy`, `vip`)	**Urgent**
F-PRX-05	Proximité live vs historique	Proximité live (Redis) pour le temps réel, PostGIS pour l'historique	**Urgent**

Performance proximité

Live (Redis) : `GEORADIUS` ou `GEOSEARCH` sur les positions Redis. Complexité O(N+log(M)). Réponse $<$ 50ms pour 100 000 entités.
Historique (PostGIS) : `ST_DWithin` avec index GiST. Réponse $<$ 200ms sur des millions de points.

5.3 Géofencing

ID	Fonctionnalité	Description	Priorité
F-GEO-01	Création de zone	Définir une zone géographique (polygone ou cercle)	**Urgent**
F-GEO-02	Zones par plateforme	Chaque plateforme gère ses propres zones	**Urgent**
F-GEO-03	Détection entrée/sortie	À chaque mise à jour de position, vérifier si l'entité entre ou sort d'une zone	**Urgent**
F-GEO-04	Événement geofence	Émettre un événement RabbitMQ lors d'une entrée ou sortie	**Urgent**
F-GEO-05	Zones actives/inactives	Activer/désactiver une zone sans la supprimer	Normale
F-GEO-06	Métadonnées zone	Tags et metadata libres sur les zones	Normale

Scalabilité du géofencing — Architecture event-driven

Problème : vérifier 100 000 positions contre 10 000 zones en polling = 1 milliard de checks par cycle. Non scalable.
Solution : géofencing event-driven en 3 couches :
Couche 1 — Cache zones en mémoire : au démarrage et à chaque modification, les zones actives sont chargées dans un cache mémoire local (par instance NestJS). Structure : R-tree ou quadtree pour lookup spatial O(log N).
Couche 2 — Vérification à chaque position update : quand une position arrive (WebSocket ou REST), le service vérifie **uniquement cette position** contre les zones pertinentes via le spatial index en mémoire. Coût : O(log Z) où Z = nombre de zones (pas O(Z)).
Couche 3 — État précédent en Redis : pour chaque entité, Redis stocke la liste des zones dans lesquelles elle se trouve actuellement (`geo:fences:{entity_id}`). La comparaison ancien état / nouvel état détermine les entrées et sorties.

Résultat : chaque mise à jour de position coûte O(log Z) au lieu de O(Z). Pour 10 000 zones et 20 000 updates/seconde : ~280 000 lookups/s (faisable sur une seule instance).
Invalidation cache : quand une zone est créée/modifiée/supprimée, un message Redis pub/sub informe toutes les instances de recharger la zone concernée (pas tout le cache).

5.4 Trajets

ID	Fonctionnalité	Description	Priorité
F-TRJ-01	Démarrer un trajet	Créer un trajet avec point de départ	Urgent
F-TRJ-02	Enregistrer les positions	Les positions envoyées pendant un trajet sont liées au trajet	Urgent
F-TRJ-03	Terminer un trajet	Clôturer le trajet avec point d'arrivée et métriques (distance, durée)	Urgent
F-TRJ-04	Consulter un trajet	Restituer le tracé complet (polyline) avec métriques	Urgent
F-TRJ-05	Historique des trajets	Liste des trajets d'une entité, filtrée par date	Urgent
F-TRJ-06	Distance calculée	Distance totale du trajet calculée à partir des points GPS	Urgent

Machine à états des trajets (v1.1)

```
[creation via POST /geo/trips] --> active
active --> completed    (POST /geo/trips/{id}/end)
active --> cancelled    (annulation explicite ou timeout admin)
```

États terminaux : `completed`, `cancelled`. Aucune transition sortante.
Règles :
- Un seul trajet actif par `(platform_id, entity_id)` à tout moment.
- Les positions GPS peuvent exister **hors trajet**. En l'absence de trajet actif, les positions sont stockées dans `position_history` avec `trip_id = NULL`.
- La création d'un trajet est une décision **explicite** via `POST /geo/trips`, pas une détection automatique.
- Si l'entité passe offline pendant un trajet actif, le trajet reste `active` jusqu'à clôture explicite ou timeout configurable par la plateforme.

5.5 Distance & ETA

ID	Fonctionnalité	Description	Priorité
F-ETA-01	Distance vol d'oiseau	Calcul interne Haversine (aucun provider externe)	Urgent
F-ETA-02	Distance routière	Via provider externe (Google Maps, OSRM)	Urgent
F-ETA-03	ETA	Temps estimé d'arrivée via provider externe	Urgent
F-ETA-04	Multi-provider	Adapter pattern : ajouter un provider = ajouter un adapter	Urgent
F-ETA-05	Cache ETA	Résultats cachés dans Redis (TTL 60s) pour éviter les appels répétés	Urgent
F-ETA-06	Fallback	Si le provider principal est down, fallback vers Haversine + estimation	Normale

Architecture multi-provider ETA/géocodage

Chaque provider est implémenté comme un adapter derrière une interface commune :

- `getDistance(origin, destination)` → `{distance_m, duration_s, polyline?}`
- `getETA(origin, destination)` → `{eta_seconds, eta_datetime}`
- `geocode(address)` → `{lat, lng, formatted_address}`
- `reverseGeocode(lat, lng)` → `{address, city, country}`

Providers supportés au MVP :

Provider	Type	Usage
Interne (Haversine)	Gratuit	Distance vol d'oiseau, fallback, rapide
OSRM	Gratuit (self-hosted)	Distance/ETA routière, open so
Google Maps	Payant	Distance/ETA précise, géocoda temps réel
Nominatim	Gratuit	Géocodage (adresse ↔ coordon OpenStreetMap

5.6 Géocodage

ID	Fonctionnalité	Description	Priorité
F-GCD-01	Géocodage direct	Adresse → coordonnées via provider externe	**Urgent**
F-GCD-02	Géocodage inverse	Coordonnées → adresse via provider externe	**Urgent**
F-GCD-03	Cache géocodage	Résultats cachés dans Redis (TTL 24h)	**Urgent**
F-GCD-04	Multi-provider	Adapter pattern (Nominatim, Google Maps)	**Urgent**

5.7 Administration

ID	Fonctionnalité	Description	Priorité
F-ADM-01	Stats plateforme	Entités actives, positions/s, trajets, zones	Normale
F-ADM-02	Config plateforme	Fréquence, TTL, providers actifs, provider ETA prioritaire	**Urgent**
F-ADM-03	Monitoring providers	Latence, taux d'erreur par provider ETA/géocodage	Normale

6 Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Latence position update → diffusion	< 200ms
NF-02	Performance	Recherche proximité live	< 50ms (P95)
NF-03	Performance	Recherche proximité historique	< 200ms (P95)
NF-04	Performance	Throughput position updates	20 000 msg/s (100K entités à 5s)
NF-05	Disponibilité	Uptime	99.9%
NF-06	Sécurité	Auth obligatoire (WebSocket et REST)	JWT ou S2S
NF-07	Sécurité	Rate limiting	Configurable par plateforme
NF-08	Scalabilité	Horizontal	Stateless, multi-instance, Redis coordonne
NF-09	Scalabilité	PostGIS partitionnement	Par mois sur position_history
NF-10	Observabilité	Logs structurés	JSON, request ID
NF-11	Observabilité	Health check	`GET /geo/health`
NF-12	Données	Timestamps	TIMESTAMPTZ
NF-13	Données	Coordonnées	GEOGRAPHY(Point, 4326) / WGS84
NF-14	Traitement	Batch flush PostGIS	Toutes les 10–30s
NF-15	Traitement	Maintenance : offline detection, batch flush PostGIS, cleanup buffers Redis, rotation jobs	Cron NestJS natif

7 Architecture Technique

7.1 Vue d'ensemble

Le service Géolocalisation est un microservice temps réel, déployé en conteneur Docker. Il expose une API REST + WebSocket. Il utilise Redis pour les données live et PostgreSQL/PostGIS pour l'historique et les requêtes spatiales.

7.2 Stack Technique

Composant	Technologie
Runtime	Node.js 20 LTS
Framework	NestJS
WebSocket	Socket.IO (NestJS Gateway) + Redis Adapter (multi-instance)
API REST	NestJS Controllers
Base de données	PostgreSQL 15+ avec extension PostGIS 3.x
ORM	TypeORM (support natif PostGIS)
Cache & Live data	Redis 7+ (GEO commands, pub/sub, cache ETA/géocodage)
Message Broker	RabbitMQ (événements geofence, position batch)
Spatial index (mémoire)	rbush (R-tree JavaScript) pour géofencing
Conteneurisation	Docker + Docker Compose
CI/CD	GitHub Actions

Justification NestJS (vs Django)

Le service Géolocalisation est I/O-intensif avec des milliers de connexions WebSocket simultanées et des flux GPS à haute fréquence. Node.js/NestJS est choisi pour les mêmes raisons que le Service Média (NestJS) et le Service Chat (WebSocket natif) :

- Event loop non bloquant : milliers de connexions simultannées sans threads
- Socket.IO natif avec Redis adapter pour le scaling horizontal
- Bibliothèques spatiales performantes (rbush, turf.js)

L'hétérogénéité de stack est acceptée en microservices quand elle est justifiée par des besoins techniques spécifiques.

7.3 Dépendances & Règles d'Architecture

Service	Type	Usage
Service Auth	Interne	Validation JWT (REST et WebSocket handshake). S2S pour appels inter-services.
Service Users	Interne	Vérification statut actif et permissions métier indirectes. **Non appelé sur chaque update GPS.** Lookup uniquement à la création d'une `tracked_entity` ou sur demande explicite. Cache Redis TTL 5 min.
Service Notification	Interne	Alertes géofencing (entrée/sortie de zone).
Google Maps API	Externe	ETA, distance routière, géocodage (payant).
OSRM	Externe	ETA, distance routière (self-hosted, gratuit).
Nominatim	Externe	Géocodage direct et inverse (gratuit).

Règle stricte de dépendance

Géolocalisation ne dépend d'aucun service métier. Il est consommé par les plateformes (MboaMove, Market) pour leurs besoins de localisation. Le couplage est unidirectionnel : les plateformes interrogent Géolocalisation, jamais l'inverse.

Exception : Géolocalisation émet des événements RabbitMQ consommés par les plateformes (geofence triggers) et Notification (alertes).

7.4 Stratégie de scalabilité

Scaling pour 100 000+ entités simultanées

WebSocket : Socket.IO avec Redis adapter. Chaque instance NestJS gère un sous-ensemble de connexions. Redis pub/sub synchronise les événements entre instances. Load balancer avec sticky sessions (ou transport polling+websocket).
Redis GEO : les commandes `GEOADD`/`GEOSEARCH` supportent des millions de points. Les positions live sont stockées dans des sets Redis par plateforme : `geo:live:{platform_id}`.
PostGIS batch writes : un buffer en mémoire accumule les positions pendant 10–30s, puis les insère en un seul `INSERT ... VALUES` batch. À 20 000 msg/s, un batch de 30s = 600 000 lignes, inséré en < 2s avec des insertions batch PostgreSQL.
PostGIS partitionnement : la table `position_history` est partitionnée par mois sur `recorded_at`. Les requêtes spatiales ne scannent que les partitions pertinentes.
Géofencing : R-tree en mémoire (rbush) avec invalidation ciblée via Redis pub/sub. O(log Z) par position update.
ETA/géocodage : cache Redis (TTL 60s ETA, 24h géocodage). Taux de cache hit > 80% en régime nominal.

7.5 Modèle de présence / offline (v1.1)

Détection de présence temps réel

Online : une entité est considérée online tant que sa clé payload Redis `geo:entity:{platform_id}:{entity_id}` n'a pas expiré (TTL configurable, défaut 30s).
Offline : dès que le TTL expire (aucune mise à jour de position reçue pendant X secondes) ou que la connexion WebSocket est fermée sans reprise :

1. La clé payload Redis expire automatiquement
2. L'entité est retirée de l'index GEO (`ZREM geo:index:{platform_id} entity_id`)
3. L'événement `geo.entity.offline` est émis **une seule fois** par transition online → offline
4. `tracked_entities.last_seen_at` est mis à jour en base

Détection : un cron NestJS (toutes les 10s) scanne les entités dont la clé payload a expiré mais qui sont encore dans l'index GEO, et effectue le nettoyage. Alternativement, Redis Keyspace Notifications (`expired` events) peuvent déclencher le processus de manière event-driven.
Règle : l'événement `geo.entity.offline` n'est émis qu'une seule fois par transition. Pas de ré-émission tant que l'entité n'est pas repassée online.

7.6 Diagramme d'architecture

```
MboaMove(mobile)  AGT-Market(web)   SALMA   Futures
    |                  |               |        |
[WebSocket]        [REST/WS]       [REST]   [REST/WS]
    |                  |               |        |
    +------+-----------+-------+-------+--------+
           |                   |
    Geolocation Service (NestJS)
      |         |          |           |          |
  Redis GEO   PostGIS    RabbitMQ   Auth(JWT)  Users
  (live)     (history)   (events)
      |
```

```
Socket.IO Redis Adapter (multi-instance sync)
    |
[Providers externes]
Google Maps | OSRM | Nominatim
```

8 Modélisation

8.1 Diagramme de Classes

Le diagramme de classes complet est fourni en annexe au format PlantUML (voir section 12). Il couvre les entités : `TrackedEntity`, `PositionHistory`, `Trip`, `TripPoint`, `Geofence`, `GeofenceEvent` et `PlatformGeoConfig`.

8.2 MLD — Modèle Logique de Données

8.2.1 Table `tracked_entities`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme
entity_type	VARCHAR(20)	NOT NULL	user, vehicle, asset
entity_id	UUID	NOT NULL	Référence externe (`users_auth.id` ou autre)
tags	JSONB	DEFAULT '[]'	Tags métier libres (`["available","vip"]`)
is_active	BOOLEAN	DEFAULT true	Tracking actif
last_seen_at	TIMESTAMPTZ	NULL	Dernière position connue
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(platform_id, entity_type, entity_id)`.

8.2.2 Table `position_history`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
entity_id	UUID	FK → tracked_entities	Entité trackée
location	GEOGRAPHY(Point, 4326)	NOT NULL	Coordonnées WGS84
altitude	FLOAT	NULL	Altitude en mètres
heading	FLOAT	NULL	Direction (0–360°)
speed	FLOAT	NULL	Vitesse en m/s
accuracy	FLOAT	NULL	Précision GPS en mètres
trip_id	UUID	FK → trips, NULL	Trajet associé (NULL si hors trajet)
recorded_at	TIMESTAMPTZ	NOT NULL	Horodatage de la mesure GPS
created_at	TIMESTAMPTZ	NOT NULL	Date d'insertion en base

Index : GiST sur `location`. B-tree sur `(entity_id, recorded_at)`.

Partitionnement : par mois sur `recorded_at`.

8.2.3 Table trips

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
entity_id	UUID	FK → tracked_entities	Entité
platform_id	UUID	NOT NULL	Plateforme
status	VARCHAR(20)	NOT NULL	active, completed, cancelled
start_location	GEOGRAPHY(Point, 4326)	NOT NULL	Point de départ
end_location	GEOGRAPHY(Point, 4326)	NULL	Point d'arrivée
start_address	VARCHAR(255)	NULL	Adresse de départ (géocodée)
end_address	VARCHAR(255)	NULL	Adresse d'arrivée
distance_meters	FLOAT	DEFAULT 0	Distance totale calculée
duration_seconds	INT	DEFAULT 0	Durée totale
metadata	JSONB	NULL	Données métier opaques
started_at	TIMESTAMPTZ	NOT NULL	Début
ended_at	TIMESTAMPTZ	NULL	Fin
created_at	TIMESTAMPTZ	NOT NULL	Création

8.2.4 Table geofences

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme
name	VARCHAR(100)	NOT NULL	Nom de la zone
fence_type	VARCHAR(10)	NOT NULL	polygon ou circle
geometry	GEOGRAPHY(Polygon, 4326)	NULL	Polygone (si polygon)
center	GEOGRAPHY(Point, 4326)	NULL	Centre (si circle)
radius_meters	FLOAT	NULL	Rayon (si circle)
tags	JSONB	DEFAULT '[]'	Tags libres
metadata	JSONB	NULL	Données complémentaires
is_active	BOOLEAN	DEFAULT true	Zone active
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(platform_id, name)`.

Index : GiST sur `geometry` et `center`.

8.2.5 Table geofence_events

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
geofence_id	UUID	FK → geofences	Zone concernée
entity_id	UUID	FK → tracked_entities	Entité
event_type	VARCHAR(10)	NOT NULL	enter ou exit
location	GEOGRAPHY(Point, 4326)	NOT NULL	Position au moment de l'événement
recorded_at	TIMESTAMPTZ	NOT NULL	Horodatage

8.2.6 Table platform_geo_configs

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	UNIQUE, NOT NULL	Plateforme
update_interval_seconds	INT	DEFAULT 5	Fréquence de mise à jour (secondes)
position_ttl_seconds	INT	DEFAULT 30	TTL Redis (offline après X secondes sans update)
batch_flush_seconds	INT	DEFAULT 15	Intervalle de flush batch PostGIS
default_eta_provider	VARCHAR(30)	DEFAULT 'haversine'	Provider ETA par défaut
default_geocode_provider	VARCHAR(30)	DEFAULT 'nominatim'	Provider géocodage par défaut
max_proximity_radius_km	FLOAT	DEFAULT 50	Rayon max autorisé pour proximité
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

9 Contrats d'API

Base URL : `/api/v1`. Tous les endpoints REST retournent du JSON. Auth : `Authorization: Bearer <token>`.

9.1 Health Check

9.1.1 GET /geo/health

Response 200 :

```
{
  "status": "healthy",
  "database": "ok",
```

```
  "postgis": "ok",
  "redis": "ok",
  "rabbitmq": "ok",
  "active_connections": 1245,
  "version": "1.2.0"
}
```

9.2 Position (REST)

9.2.1 POST /geo/positions

Envoi d'une position (REST, usage ponctuel). Auth : Bearer ou S2S.

Request body :

```
{
  "platform_id": "uuid",
  "entity_type": "user",
  "entity_id": "user-uuid",
  "latitude": 3.8480,
  "longitude": 11.5021,
  "heading": 45.0,
  "speed": 12.5,
  "accuracy": 10.0,
  "tags": ["available"],
  "recorded_at": "2026-04-05T10:00:00Z"
}
```

Response 200 :

```
{
  "entity_id": "user-uuid",
  "status": "updated",
  "geofence_events": [
    { "geofence_id": "zone-uuid", "name": "Zone Centre", "event": "enter
      " }
  ]
}
```

Codes : 200, 400, 401, 429 Rate limited.

9.2.2 GET /geo/positions/{entityId}

Dernière position connue (depuis Redis). Auth : Bearer ou S2S.

Response 200 :

```
{
  "entity_id": "user-uuid",
  "latitude": 3.8480,
  "longitude": 11.5021,
  "heading": 45.0,
  "speed": 12.5,
  "tags": ["available"],
  "recorded_at": "2026-04-05T10:00:00Z",
  "is_online": true
}
```

Codes : 200, 404 Entity not found or offline.

9.3 Position (WebSocket)

9.3.1 Connexion WebSocket

Endpoint : `ws://host/geo`. Auth : token JWT passé en query param ou header lors du handshake.

Événements client → serveur :

Événement	Payload
`position:update`	`{platform_id, entity_type, entity_id, latitude, longitude, heading, speed, accuracy, tags, recorded_at}`
`position:subscribe`	`{platform_id, entity_ids: ["uuid1", "uuid2"]}` — s'abonner aux positions d'autres entités
`position:unsubscribe`	`{entity_ids: ["uuid1"]}`

Événements serveur → client :

Événement	Payload
`position:updated`	`{entity_id, latitude, longitude, heading, speed, recorded_at}` — position d'une entité suivie
`position:offline`	`{entity_id}` — entité passée offline (TTL expiré)
`geofence:trigger`	`{entity_id, geofence_id, name, event: "enter"|"exit", location}`

Règles d'autorisation WebSocket (v1.1)

L'abonnement WebSocket aux positions d'autres entités n'est **jamais libre**.
Cas 1 — Suivi de sa propre position : autorisé. Un client JWT peut toujours écouter ses propres mises à jour.
Cas 2 — Suivi d'autres entités (frontend métier) : interdit en accès direct JWT sauf si :

- permission explicite accordée par le backend produit de la plateforme (ex : un passager MboaMove suit son chauffeur assigné) ;
- ou token S2S émis par le backend de la plateforme avec la liste des `entity_ids` autorisés.

Cas 3 — Abonnement massif (toutes les entités d'une plateforme) : réservé aux appels S2S ou aux rôles admin plateforme. Jamais accessible en JWT standard.
Implémentation : lors d'un `position:subscribe`, le service vérifie que chaque `entity_id` demandé est soit l'appelant lui-même, soit présent dans une liste d'autorisations fournie par le backend produit (via un cache Redis `geo:auth:{user_id}:{platform_id}` avec TTL 5 min).

9.4 Proximité

9.4.1 GET /geo/proximity

Recherche de proximité live. Auth : Bearer ou S2S.

Query params : `?platform_id=uuid&latitude=3.85&longitude=11.50&radius_km=5&limit=10&tags=availa`

Response 200 :

```
{
  "results": [
    {
```

```
      "entity_id": "uuid-1",
      "latitude": 3.8490,
      "longitude": 11.5030,
      "distance_meters": 320,
      "tags": ["available"],
      "recorded_at": "2026-04-05T10:00:05Z"
    },
    {
      "entity_id": "uuid-2",
      "latitude": 3.8510,
      "longitude": 11.5050,
      "distance_meters": 580,
      "tags": ["available", "vip"],
      "recorded_at": "2026-04-05T10:00:03Z"
    }
  ],
  "total": 2
}
```

Codes : 200, 400 Radius exceeds max, 401.

9.5 Trajets

9.5.1 POST /geo/trips

Démarrer un trajet.

Request body :

```
{
  "platform_id": "uuid",
  "entity_id": "user-uuid",
  "start_latitude": 3.8480,
  "start_longitude": 11.5021,
  "metadata": { "ride_id": "ride-uuid" }
}
```

Codes : 201, 400, 409 Active trip already exists.

9.5.2 POST /geo/trips/{tripId}/end

Terminer un trajet.

Request body :

```
{
  "end_latitude": 3.8700,
  "end_longitude": 11.5200
}
```

Response 200 :

```
{
  "trip_id": "trip-uuid",
  "status": "completed",
  "distance_meters": 4520,
  "duration_seconds": 1230,
  "start_address": "Carrefour Bastos, Yaounde",
  "end_address": "Marche Central, Yaounde"
```

```
}
```

Codes : 200, 404, 409 Trip already ended.

9.5.3 GET /geo/trips/{tripId}

Détail d'un trajet avec polyline.

Codes : 200, 404.

9.5.4 GET /geo/trips

Historique des trajets. Query params : `?entity_id&platform_id&from_date&to_date&page&limit`.

Codes : 200, 401.

9.6 Géofences

9.6.1 POST /geo/geofences

Création d'une zone. Auth : admin plateforme.

Request body (polygone) :

```
{
  "platform_id": "uuid",
  "name": "Zone Centre Yaounde",
  "fence_type": "polygon",
  "coordinates": [
    [11.49, 3.84], [11.52, 3.84],
    [11.52, 3.87], [11.49, 3.87],
    [11.49, 3.84]
  ],
  "tags": ["delivery_zone"]
}
```

Request body (cercle) :

```
{
  "platform_id": "uuid",
  "name": "Rayon Aeroport Nsimalen",
  "fence_type": "circle",
  "center_latitude": 3.7200,
  "center_longitude": 11.5533,
  "radius_meters": 5000,
  "tags": ["airport_zone"]
}
```

Codes : 201, 400, 409 Name exists.

9.6.2 GET /geo/geofences

Liste des zones. Query params : `?platform_id&is_active&tags`.

Codes : 200.

9.6.3 PUT /geo/geofences/{geofenceId}

Modification. Déclenche invalidation cache mémoire via Redis pub/sub.

Codes : 200, 400, 404.

9.6.4 DELETE /geo/geofences/{geofenceId}

Suppression (soft : `is_active = false`). Déclenche une invalidation ciblée via Redis pub/sub pour rechargement de l'index mémoire sur toutes les instances.

Codes : 200, 404.

Invalidation cache geofence (v1.1)

Toute création, modification, désactivation (`is_active = false`) ou suppression logique d'une geofence déclenche une invalidation ciblée via Redis pub/sub. Chaque instance NestJS recharge uniquement la zone concernée dans son R-tree local, pas l'ensemble du cache.

9.7 Distance & ETA

9.7.1 GET /geo/distance

Query params : `?origin_lat&origin_lng&dest_lat&dest_lng&provider=google_maps`

Response 200 :

```
{
  "provider": "google_maps",
  "distance_meters": 4520,
  "duration_seconds": 1230,
  "cached": false
}
```

Codes : 200, 400, 503 Provider unavailable.

9.7.2 GET /geo/eta

Query params : `?origin_lat&origin_lng&dest_lat&dest_lng&provider=osrm`

Response 200 :

```
{
  "provider": "osrm",
  "eta_seconds": 1230,
  "eta_datetime": "2026-04-05T10:20:30Z",
  "cached": true
}
```

Codes : 200, 400, 503.

9.8 Géocodage

9.8.1 GET /geo/geocode

Query params : `?address=Carrefour+Bastos+Yaounde&provider=nominatim`

Response 200 :

```
{
  "latitude": 3.8845,
  "longitude": 11.5027,
  "formatted_address": "Carrefour Bastos, Yaounde, Cameroun",
  "provider": "nominatim",
  "cached": true
}
```

Codes : 200, 400, 404 Address not found, 503.

9.8.2 GET /geo/reverse-geocode

Query params : `?latitude=3.8845&longitude=11.5027&provider=nominatim`

Codes : 200, 400, 503.

9.9 Configuration Plateforme

9.9.1 PUT /geo/config/{platformId}

Auth : admin plateforme ou admin global.

Request body :

```
{
  "update_interval_seconds": 5,
  "position_ttl_seconds": 30,
  "batch_flush_seconds": 15,
  "default_eta_provider": "osrm",
  "default_geocode_provider": "nominatim",
  "max_proximity_radius_km": 50
}
```

Codes : 200, 400, 403.

9.9.2 GET /geo/config/{platformId}

Codes : 200, 404.

9.10 Administration

9.10.1 GET /geo/admin/stats

Query params : `?platform_id`. Auth : admin.

Response 200 :

```
{
  "active_entities": 12450,
  "active_websockets": 8900,
  "positions_per_second": 1780,
  "active_trips": 340,
  "active_geofences": 125,
  "geofence_events_24h": 45200
}
```

Codes : 200, 403.

9.10.2 DELETE /geo/by-user/{userId}

Purge RGPD. Auth : S2S. Supprime les `tracked_entities`. Les positions historiques et trajets sont anonymisés par remplacement de `entity_id` par un identifiant technique d'anonymisation irréversible (UUID généré, sans lien vers `users_auth.id`). Les lignes historiques sont conservées pour l'analytics spatiale.

Codes : 200, 401, 404.

10 Contrats Inter-Services

10.1 Géolocalisation → Auth (JWT / S2S)

Paramètre	Valeur
Endpoint JWT	`GET /auth/verify-token`. Valide au WebSocket handshake + chaque appel REST. Cache Redis TTL 30s.
Endpoint S2S	`POST /auth/s2s/introspect`

10.2 Géolocalisation → Notification (alertes)

Événement	Notification déclenchée
Geofence enter	Push : « Vous êtes entré dans la zone X » (si config plateforme l'exige)
Geofence exit	Push : « Vous avez quitté la zone X »

10.3 Géolocalisation — Événements RabbitMQ sortants

Événement publié	Payload
`geo.position.updated`	`entity_id`, `platform_id`, `latitude`, `longitude`, `speed`, `heading`, `tags`, `recorded_at`
`geo.geofence.enter`	`entity_id`, `geofence_id`, `geofence_name`, `platform_id`, `location`, `recorded_at`
`geo.geofence.exit`	Idem
`geo.trip.started`	`trip_id`, `entity_id`, `platform_id`, `start_location`
`geo.trip.ended`	`trip_id`, `entity_id`, `platform_id`, `distance_meters`, `duration_seconds`
`geo.entity.offline`	`entity_id`, `platform_id`, `last_seen_at`

Routage des événements

Les événements sont publiés sur un exchange RabbitMQ de type `topic`. Les consommateurs s'abonnent selon leur besoin :

- MboaMove écoute `geo.position.updated` pour le dispatch
- AGT-Market écoute `geo.geofence.*` pour la livraison
- Notification écoute `geo.geofence.*` pour les alertes push

10.4 Users → Géolocalisation (Purge RGPD)

Paramètre	Valeur
Endpoint	`DELETE /api/v1/geo/by-user/{userId}`
Auth	Token S2S
Données anonymisées	`tracked_entities` supprimées. Dans `position_history` et `trips`, `entity_id` est remplacé par un UUID technique d'anonymisation irréversible (pas NULL, FK conservée vers une entité anonyme). Aucune ré-identification possible.
Séquence	Avant Auth purge

11 Matrice d'Autorisation

Endpoint	JWT	S2S	Admin Plat.	Admin Global
POST /geo/positions	✓	✓		
GET /geo/positions/{entityId}	✓	✓	✓	✓
WebSocket /geo	✓			
GET /geo/proximity	✓	✓	✓	✓
POST /geo/trips	✓	✓		
POST /geo/trips/{id}/end	✓	✓		
GET /geo/trips/{id}	✓	✓	✓	✓
GET /geo/trips	✓	✓	✓	✓
POST /geo/geofences			✓	✓
GET /geo/geofences	✓	✓	✓	✓
PUT /geo/geofences/{id}			✓	✓
DELETE /geo/geofences/{id}			✓	✓
GET /geo/distance	✓	✓		
GET /geo/eta	✓	✓		
GET /geo/geocode	✓	✓		
GET /geo/reverse-geocode	✓	✓		
PUT /geo/config/{platformId}			✓	✓
GET /geo/config/{platformId}			✓	✓
GET /geo/admin/stats			✓	✓
DELETE /geo/by-user/{userId}		✓		✓

12 Recommandations & Évolutions

- **Heatmap** : visualisation de densité de positions sur une zone (utile pour analytics MboaMove/Market).
- **Clustering** : regroupement d'entités proches pour l'affichage carte à faible zoom.
- **Replay de trajet** : rejouer un trajet en temps accéléré (utile pour audit, réclamations).
- **Alertes vitesse** : détecter les excès de vitesse dans les zones géofencées.
- **Isochrones** : zones accessibles en N minutes depuis un point.
- **Offline-first mobile** : buffer GPS côté mobile avec sync à la reconnexion.
- **Monitoring** : métriques Prometheus (positions/s, connexions WS, latence proximité, cache hit rate ETA).
- **Versioning API** : préfixe `/api/v1`. Breaking changes → `/api/v2`.

Annexes

Glossaire

Terme	Définition
PostGIS	Extension spatiale de PostgreSQL pour les données géographiques
WGS84	Système de coordonnées géodésiques mondial (SRID 4326)
GiST	Generalized Search Tree — index PostgreSQL optimisé pour les données spatiales
Geofence	Zone géographique virtuelle déclenchant des alertes à l'entrée/sortie
R-tree	Structure de données spatiale pour la recherche par rectangle englobant
Haversine	Formule de calcul de distance entre deux points sur une sphère
ETA	Estimated Time of Arrival — temps estimé d'arrivée
OSRM	Open Source Routing Machine — moteur de routage open source
Geocoding	Conversion adresse → coordonnées (et inverse)
Socket.IO	Bibliothèque WebSocket avec fallback polling et reconnexion automatique
Sticky session	Affinité de session entre un client et une instance serveur
S2S	Service-to-Service
DLQ	Dead Letter Queue

Code PlantUML — Diagramme de Classes

Le code PlantUML ci-dessous correspond au diagramme de classes complet du service Géolocalisation v1.2. Il peut être rendu sur `plantuml.com` ou tout outil compatible.

```
@startuml Geolocation Service - Class Diagram v1.2

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "tracked_entities" as entities {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  * entity_type : VARCHAR(20)
  * entity_id : UUID
  tags : JSONB = '[]'
  is_active : BOOLEAN = true
  last_seen_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (platform_id, entity_type, entity_id)
}

entity "position_history" as positions {
  * id : UUID <<PK>>
  --
  * entity_id : UUID <<FK>>
  * location : GEOGRAPHY(Point, 4326)
  altitude : FLOAT
  heading : FLOAT
  speed : FLOAT
  accuracy : FLOAT
  trip_id : UUID <<FK>>
  * recorded_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  ..
  <<INDEX GiST>> location
  <<PARTITION>> BY RANGE (recorded_at)
}

entity "trips" as trips {
```

```
  * id : UUID <<PK>>
  --
  * entity_id : UUID <<FK>>
  * platform_id : UUID
  * status : VARCHAR(20)
  * start_location : GEOGRAPHY(Point, 4326)
  end_location : GEOGRAPHY(Point, 4326)
  start_address : VARCHAR(255)
  end_address : VARCHAR(255)
  distance_meters : FLOAT = 0
  duration_seconds : INT = 0
  metadata : JSONB
  * started_at : TIMESTAMPTZ
  ended_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
}

entity "geofences" as fences {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  * name : VARCHAR(100)
  * fence_type : VARCHAR(10)
  geometry : GEOGRAPHY(Polygon, 4326)
  center : GEOGRAPHY(Point, 4326)
  radius_meters : FLOAT
  tags : JSONB = '[]'
  metadata : JSONB
  is_active : BOOLEAN = true
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (platform_id, name)
  <<INDEX GiST>> geometry, center
}

entity "geofence_events" as geoevents {
  * id : UUID <<PK>>
  --
  * geofence_id : UUID <<FK>>
  * entity_id : UUID <<FK>>
  * event_type : VARCHAR(10)
  * location : GEOGRAPHY(Point, 4326)
  * recorded_at : TIMESTAMPTZ
}

entity "platform_geo_configs" as geoconfig {
  * id : UUID <<PK>>
  --
  * platform_id : UUID <<UQ>>
  update_interval_seconds : INT = 5
  position_ttl_seconds : INT = 30
  batch_flush_seconds : INT = 15
  default_eta_provider : VARCHAR(30) = 'haversine'
  default_geocode_provider : VARCHAR(30) = 'nominatim'
  max_proximity_radius_km : FLOAT = 50
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

entities ||--o{ positions    : "0..* positions"
entities ||--o{ trips        : "0..* trips"
entities ||--o{ geoevents    : "0..* geofence events"

trips ||--o{ positions       : "0..* trip points"

fences ||--o{ geoevents      : "0..* events"

@enduml
```

AG Technologies — Document confidentiel à usage interne exclusivement.
Geolocation Service CDC v1.2 — Avril 2026